UNIVERSIDADE PAULISTA – UNIP
CURSO DE CIÊNCIA DA COMPUTAÇÃO

ALUISIO PEREIRA ALVES N135891
ENZO ORLANDI GOMES G788EJ5
ERICK DE BRITO CARVALHO G78HED3
KAYKY CRESPO DOS SANTOS G839226

CINTO INTELIGENTE COM IoT E INTELIGÊNCIA ARTIFICIAL PARA MONITORAMENTO DE ATIVIDADES, SINAIS VITAIS E LOCALIZAÇÃO INDOOR

Sistema de Detecção de Possíveis Quedas

Guia do Usuário e Manual de Operação

SÃO PAULO
2026

ALUISIO PEREIRA ALVES – RA N135891
ENZO ORLANDI GOMES – RA G788EJ5
ERICK DE BRITO CARVALHO – RA G78HED3
KAYKY CRESPO DOS SANTOS – RA G839226

CINTO INTELIGENTE COM IoT E INTELIGÊNCIA ARTIFICIAL PARA MONITORAMENTO DE ATIVIDADES, SINAIS VITAIS E LOCALIZAÇÃO INDOOR

Sistema de Detecção de Possíveis Quedas

Guia do Usuário e Manual de Operação do conjunto prototipado, apresentado ao Curso de Ciência da Computação da Universidade Paulista – UNIP como entregável do Trabalho de Conclusão de Curso, na vertente SmartCity – Cidades Inteligentes e Infraestrutura Urbana.

Orientador: Prof. Marco Gomes

SÃO PAULO

2026

SUMÁRIO

Seção 1 — Identificação do Ecossistema e Vertente 4
1.1 Identificação 4
1.2 Enquadramento na vertente SmartCity 4
1.3 Diagrama de arquitetura de alto nível 6
1.4 Fluxo de funcionamento 7
Seção 2 — Manual de Instalação e Montagem do Hardware 8
2.1 Lista de materiais (BOM) 8
2.2 Pinagem (pinout) 9
2.3 Diagrama esquemático e montagem 10
2.4 Especificações de alimentação 13
2.5 Ordem de montagem 19
2.6 Fixação mecânica e encapsulamento 22
Seção 3 — Guia de Instalação e Configuração do Software 23
3.1 Requisitos mínimos 23
3.2 Configuração da borda (firmware) 23
3.3 Subida do servidor, backend e banco de dados 26
3.4 Aplicação cliente (painel) 30
Seção 4 — Módulo de Inteligência Artificial 31
4.1 Objetivo operacional 31
4.2 Dataset de treinamento e validação 31
4.3 Arquitetura do modelo e treinamento 32
4.4 Resultados e métricas 33
4.5 Declaração formal de uso de IA assistiva 34
Seção 5 — Manual de Operação e Diagnóstico 35
5.1 Partida e calibração 35
5.2 Operação diária 36
5.3 Resolução de problemas 38
5.4 Desligamento seguro e manutenção preventiva 39

SEÇÃO 1 — IDENTIFICAÇÃO DO ECOSSISTEMA E VERTENTE

1.1 Identificação

A solução apresentada neste guia é o Cinto Inteligente com IoT e Inteligência Artificial para Monitoramento de Atividades, Sinais Vitais e Localização Indoor, um sistema de detecção de possíveis quedas desenvolvido como Trabalho de Conclusão de Curso do Bacharelado em Ciência da Computação da Universidade Paulista (UNIP), em 2026, sob orientação do Prof. Marco Gomes.

O projeto está enquadrado na vertente SmartCity – Cidades Inteligentes e Infraestrutura Urbana.

A equipe executora é composta pelos seguintes integrantes, com as respectivas funções no projeto:

- Aluisio Pereira Alves (RA N135891) – pesquisador e redator;
- Enzo Orlandi Gomes (RA G788EJ5) – responsável pelo sistema embarcado;
- Erick de Brito Carvalho (RA G78HED3) – arquiteto de soluções e desenvolvedor de software;
- Kayky Crespo dos Santos (RA G839226) – Product Owner, gerente de projeto e Scrum Master.

1.2 Enquadramento na vertente SmartCity

O envelhecimento da população transforma o cuidado com idosos em um serviço urbano: instituições de longa permanência, unidades de saúde e programas de atendimento domiciliar dependem de equipes que não conseguem acompanhar todas as pessoas o tempo todo. Na classificação de Giffinger et al. (2007), esse domínio corresponde à dimensão de qualidade de vida (*smart living*) da cidade inteligente, e a saúde conectada é um dos principais serviços urbanos apoiados por IoT (Zanella et al., 2014; Gubbi et al., 2013).

O sistema é uma infraestrutura de sensoriamento para esses espaços, com as mesmas características das redes de telemetria urbana:

- Rede fixa de sensores: antenas instaladas no prédio captam o sinal do cinto e medem a intensidade do sinal recebido (RSSI) em cada ponto do ambiente.
- Transporte padronizado: os dados trafegam por Wi-Fi e MQTT, protocolo de publicação e assinatura usado em telemetria de cidades inteligentes.
- Processamento local: a classificação e a localização rodam no computador do prédio, sem câmeras e sem nuvem, o que reduz a latência e preserva a privacidade (LGPD).
- Serviço de alerta: a equipe de cuidado recebe o aviso em segundos, com a região do ambiente onde a pessoa caiu.
- Escalabilidade: a mesma rede pode ser replicada em vários prédios e, como evolução futura, integrada a centrais de teleassistência do município.

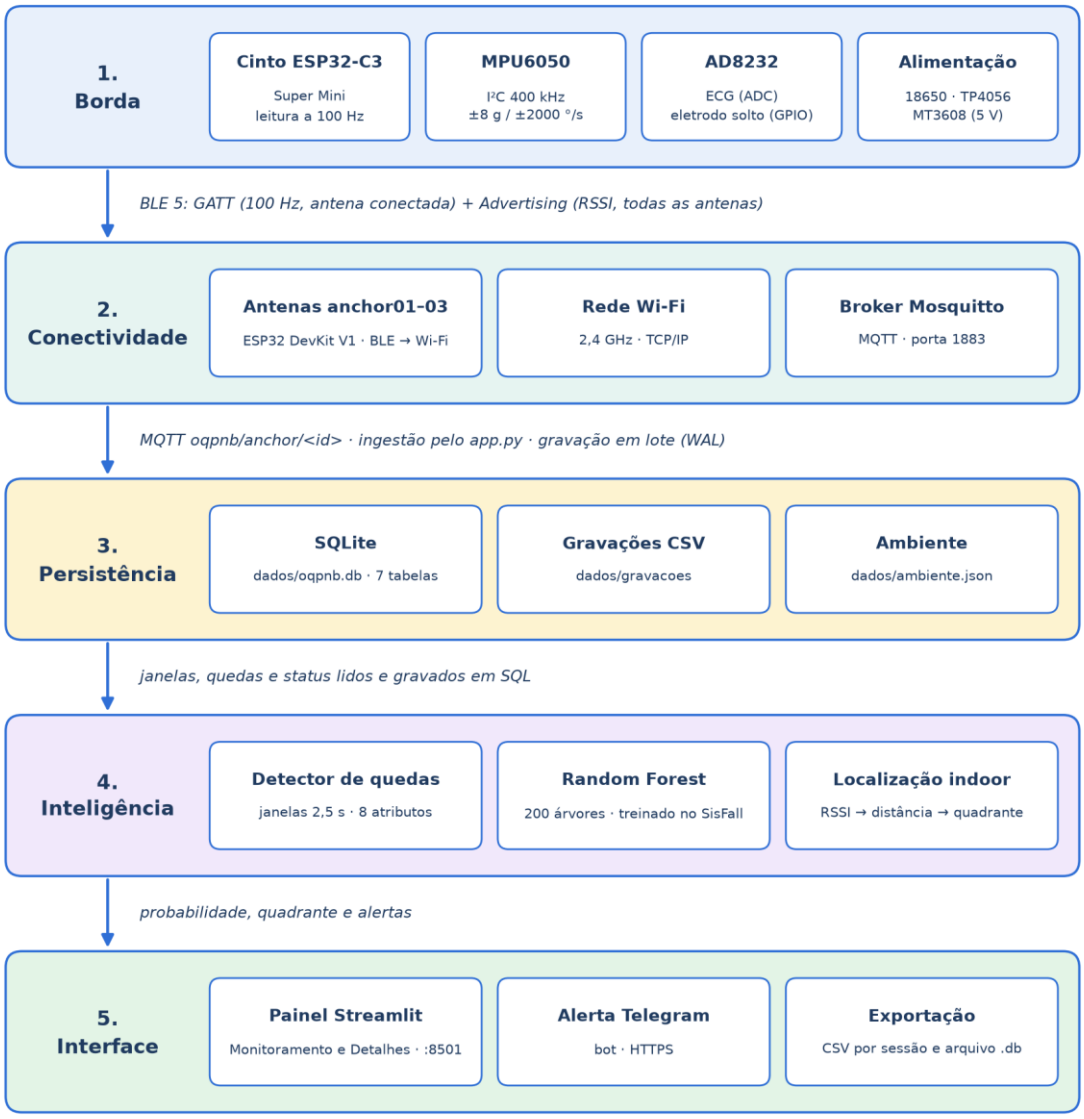
Problema atacado: a demora em perceber uma queda de pessoa idosa que vive ou permanece sozinha em ambiente interno. Muitas soluções dependem de a própria pessoa acionar um botão de emergência, o que nem sempre é possível após a queda. Outras usam GPS, pouco confiável em ambientes internos, ou geram alarmes falsos a cada movimento brusco.

Diferenciais da proposta: alerta automático, que não depende de quem caiu; privacidade, sem câmeras; e inovação frugal, com componentes de baixo custo e fáceis de encontrar (ESP32, MPU6050, AD8232), acessíveis a instituições e famílias.

Hipótese orientadora: um cinto com sensor inercial na cintura, classificado por Random Forest treinado em dataset público, detecta possíveis quedas com alta sensibilidade e, combinado às antenas BLE, informa a região aproximada do ambiente onde a queda ocorreu.

1.3 Diagrama de arquitetura de alto nível

Figura 1 – Arquitetura do ecossistema em camadas



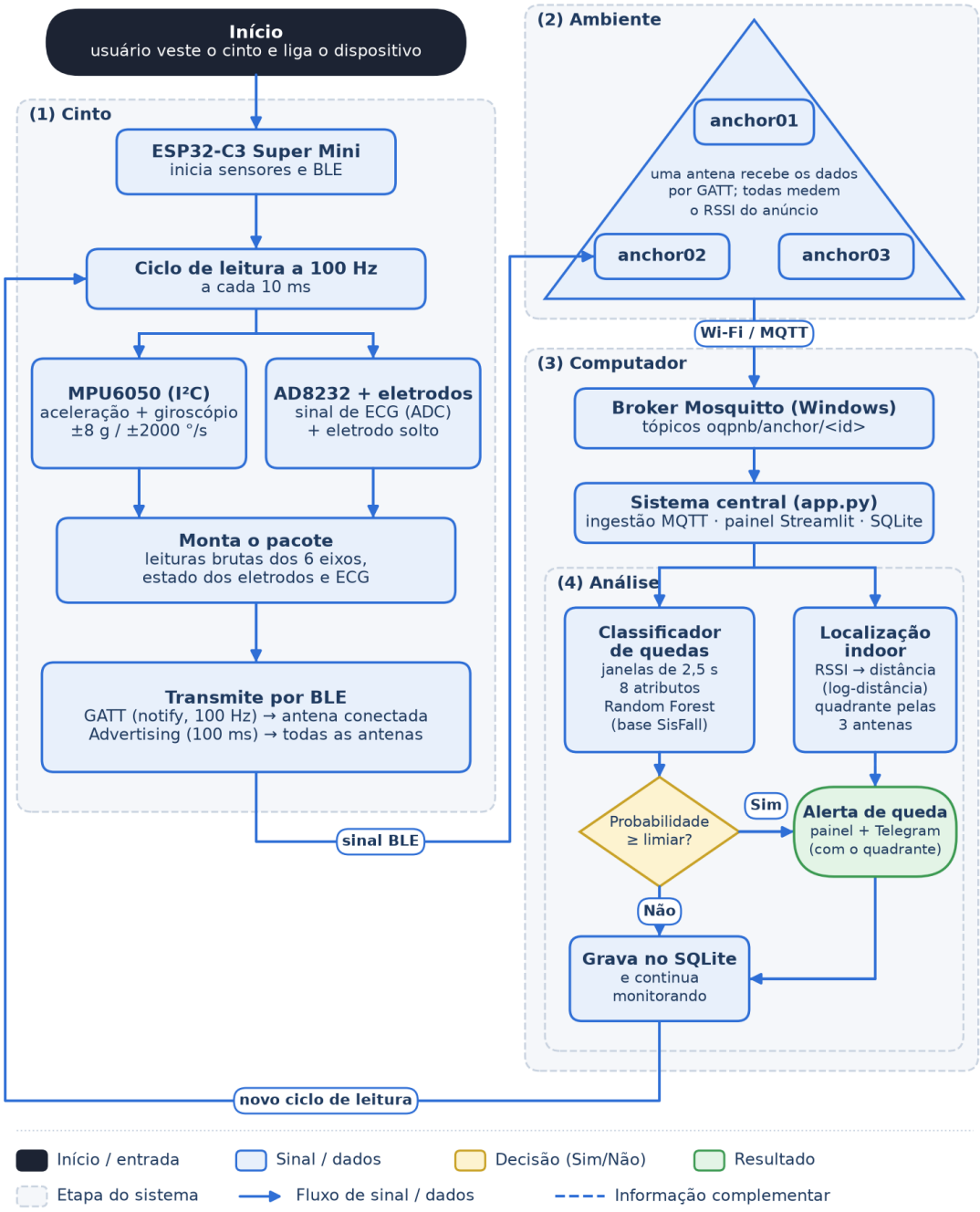
Fonte: elaborado pelos autores (2026).

Camada	Componentes	Protocolos
Borda	Cinto (ESP32-C3, MPU6050, AD8232)	I²C (MPU), ADC e GPIO (AD8232)
Conectividade	3 antenas ESP32, roteador Wi-Fi, Mosquitto	BLE 5 (GATT + Advertising), Wi-Fi 802.11 b/g/n 2,4 GHz, TCP/IP, MQTT 3.1.1
Inteligência	`detector_queda.py`, `features.py`, `localizacao.py`, modelo `modelos/modelo_mvp_8g.joblib`	—
Persistência	SQLite (`banco.py`), CSV	SQL
Interface	Painel Streamlit (`app.py`), bot do Telegram (`alerta_telegram.py`)	HTTP (navegador), HTTPS (API do Telegram)

1.4 Fluxo de funcionamento

A Figura 2 mostra o caminho de um ciclo de leitura, do sensor no cinto até o alerta.

Figura 2 – Fluxograma do funcionamento completo do sistema



Fonte: elaborado pelos autores (2026). Gerado por `DOCUMENTACAO/esquemas/gerar_fluxograma.py`.

SEÇÃO 2 — MANUAL DE INSTALAÇÃO E MONTAGEM DO HARDWARE

O conjunto tem 1 cinto e 3 antenas idênticas (`anchor01`, `anchor02`, `anchor03`).

2.1 Lista de materiais (BOM)

Cinto (1 unidade)

Qtd	Componente	Modelo / part number	Tensão de operação	Função no circuito
1	Microcontrolador	ESP32-C3 Super Mini (Espressif ESP32-C3, RISC-V, Wi-Fi + BLE 5)	5 V no pino 5V (regulador interno para 3,3 V)	Lê os sensores a 100 Hz e transmite por BLE
1	Sensor inercial	MPU6050 (módulo GY-521)	3,3 V	Aceleração e velocidade angular (±8 g / ±2000 °/s)
1	Front-end de ECG	AD8232 (módulo SparkFun ou equivalente) + cabo P2 (3,5 mm) de 3 vias	3,3 V	Sinal de ECG e detecção de eletrodo solto
3	Eletrodos	Eletrodo descartável de ECG com gel (Ag/AgCl), encaixe de pressão	—	Contato com a pele (RA, LA, RL)
1	Carregador de bateria	TP4056 com proteção (DW01A), entrada USB-C	5 V de entrada; 4,2 V de corte	Carrega as baterias e as protege
1	Conversor elevador	MT3608 (step-up)	Entrada 2–24 V; saída ajustada para 5,0 V	Eleva 3,7 V das baterias para 5 V
2	Bateria	Li-ion 18650, 3,7 V, 1000 mAh cada, em paralelo (2000 mAh no total)	3,0–4,2 V	Alimentação portátil
1	Chave	Mini chave gangorra (KCD11 ou equivalente)	—	Liga/desliga
1	LED + resistor (opcional)	LED 3 mm + resistor 330 Ω	3,3 V	Indica eletrodo solto (GPIO 10 no firmware; não aparece na montagem atual da Figura 3)
1	Caixa	Case impresso em 3D (base + tampa)	—	Proteção e fixação no cinto
2–4	Insertos e parafusos	Rosca de latão M3 + parafuso M3	—	Fixação da tampa
—	Fios	Fio de cobre	—	Ligações

Qtd	Componente	Modelo / part number	Tensão de operação	Função no circuito
		flexível (vermelho/preto para energia; cores diversas para sinal)		soldadas

Antena (por unidade; montar 3)

Qtd	Componente	Modelo / part number	Tensão de operação	Função no circuito
1	Microcontrolador	ESP32 DevKit V1 (ESP32-WROOM-32, conversor USB-serial CP2102, Micro-USB)	5 V no pino VIN	Recebe BLE do cinto, mede RSSI e publica por Wi-Fi/MQTT
1	Carregador de bateria	TP4056 com proteção, USB-C	5 V de entrada	Carrega a bateria
1	Conversor elevador	MT3608	Saída ajustada para 5,0 V	Eleva 3,7 V para 5 V
1	Bateria	Li-ion 18650, 3,7 V, 1000 mAh	3,0–4,2 V	Alimentação portátil
1	Chave	Mini chave gangorra	—	Liga/desliga
1	Caixa	Case impresso em 3D com tampa perfurada e identificação gravada	—	Proteção e fixação
2	Insertos e parafusos	Rosca de latão M3 + parafuso M3	—	Fixação da tampa
—	Fios	Fio de cobre	—	Ligações soldadas

Infraestrutura

Qtd	Item	Observação
1	Computador central	Windows 10/11 64 bits; executa broker, backend e painel
1	Roteador ou ponto de acesso Wi-Fi	Rede de 2,4 GHz (o ESP32 não opera em 5 GHz)
1	Cabo USB-C (dados)	Gravação do cinto
1	Cabo Micro-USB (dados)	Gravação das antenas

2.2 Pinagem (pinout)

Cinto — ESP32-C3 Super Mini ↔ periféricos (definida em `Arduino/cinto_mvp_v1/cinto_mvp_v1.ino`)

Pino do ESP32-C3	Pino do periférico	Sinal	Observação
3V3	MPU6050 VCC	Alimentação 3,3 V	
GND	MPU6050 GND	Terra	
GPIO 8	MPU6050 SDA	Dados I²C	I²C padrão do ESP32-C3 (`Wire.begin()` sem argumentos), 400 kHz

Pino do ESP32-C3	Pino do periférico	Sinal	Observação
GPIO 9	MPU6050 SCL	Clock I²C	Endereço do MPU: `0x68` (AD0 em GND)
3V3	AD8232 3.3V	Alimentação 3,3 V	
GND	AD8232 GND	Terra	
GPIO 4 (ADC)	AD8232 OUTPUT	Sinal de ECG analógico	ADC de 12 bits (0–4095)
GPIO 2	AD8232 LO−	Eletrodo solto (−)	Entrada digital
GPIO 3	AD8232 LO+	Eletrodo solto (+)	Entrada digital
GPIO 10	LED (ânodo, via resistor de 330 Ω)	Indicador	Aceso = eletrodo solto (opcional)
5V	MT3608 VOUT+	Alimentação 5 V	
GND	MT3608 VOUT−	Terra comum	

Cinto — alimentação

Origem	Destino
Baterias 18650 (+), em paralelo	TP4056 B+
Baterias 18650 (−)	TP4056 B−
TP4056 OUT+	Chave gangorra (terminal 1)
Chave gangorra (terminal 2)	MT3608 VIN+
TP4056 OUT−	MT3608 VIN−
MT3608 VOUT+ (5,0 V)	ESP32-C3 5V
MT3608 VOUT−	ESP32-C3 GND
Cabo P2 dos eletrodos	Conector de 3,5 mm do AD8232 (RA, LA, RL)

Antena — ESP32 DevKit V1

A antena não tem sensores; só precisa de alimentação.

Origem	Destino
Bateria 18650 (+) / (−)	TP4056 B+ / B−
TP4056 OUT+	Chave gangorra → MT3608 VIN+
TP4056 OUT−	MT3608 VIN−
MT3608 VOUT+ (5,0 V)	ESP32 VIN
MT3608 VOUT−	ESP32 GND

2.3 Diagrama esquemático e montagem

Os esquemas foram gerados a partir da pinagem do firmware pelo script `DOCUMENTACAO/esquemas/gerar_esquemas.py`. Para regenerar após alguma mudança: `pip install schemdraw` e `python DOCUMENTACAO/esquemas/gerar_esquemas.py`. Os rótulos de rede com o mesmo nome (`+5V`, `+3V3`, `GND`, `SDA`, `SCL`, `LO-`, `LO+`, `ECG_OUT`) estão ligados entre si.

Figura 3 – Disposição dos módulos no case do cinto

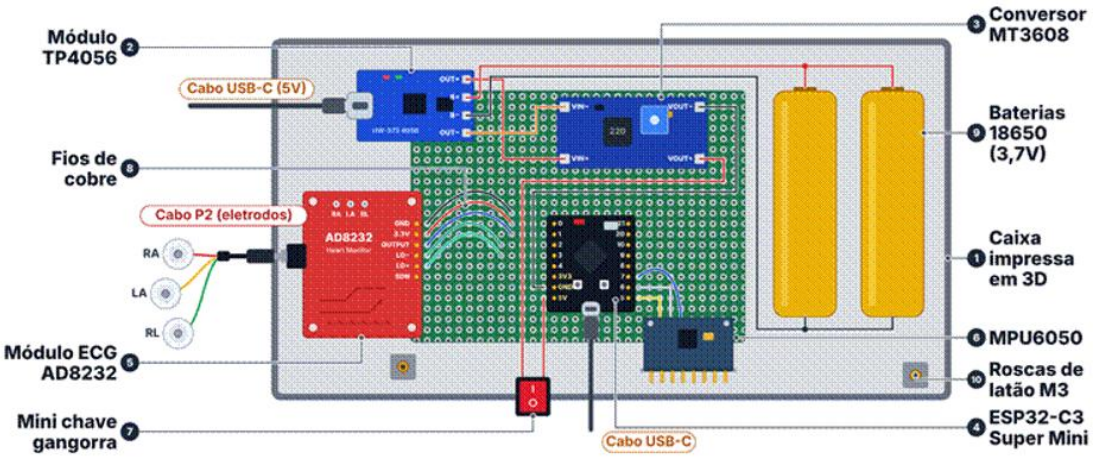


Figura 4 – Esquema elétrico do cinto

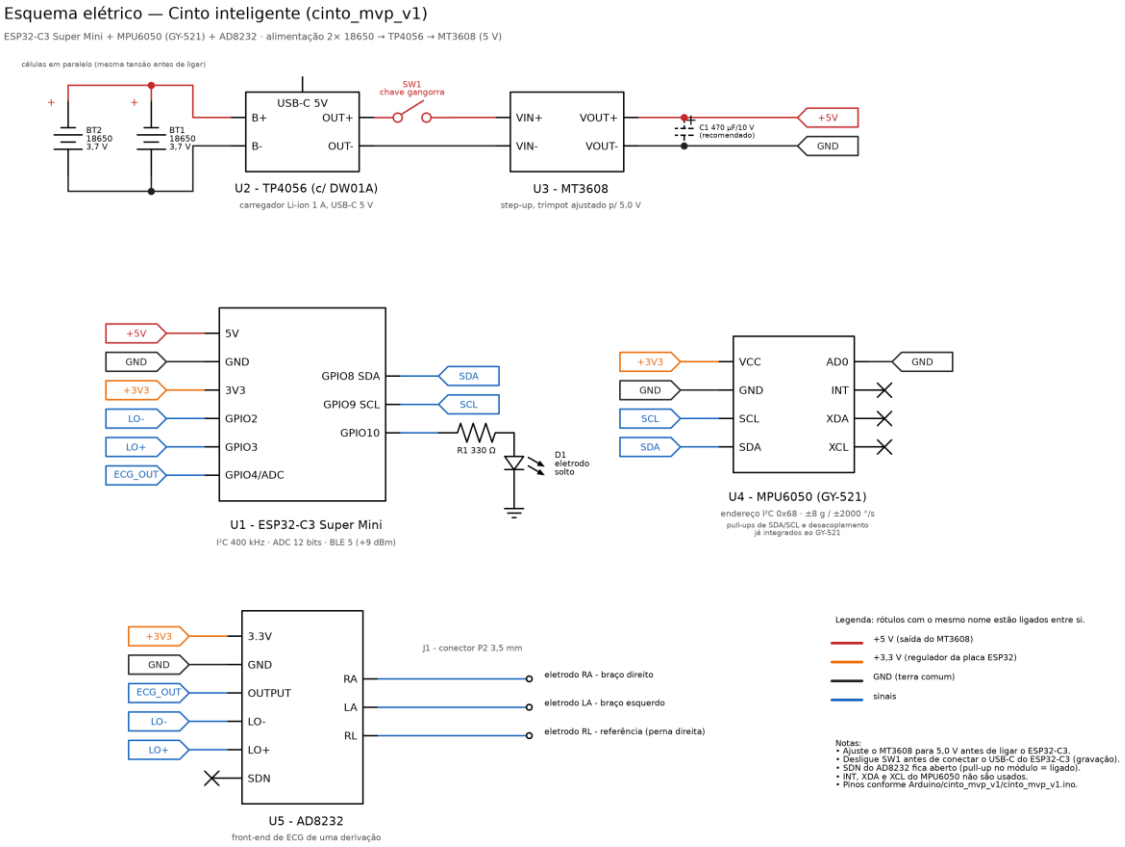


Figura 5 – Disposição dos módulos no case da antena

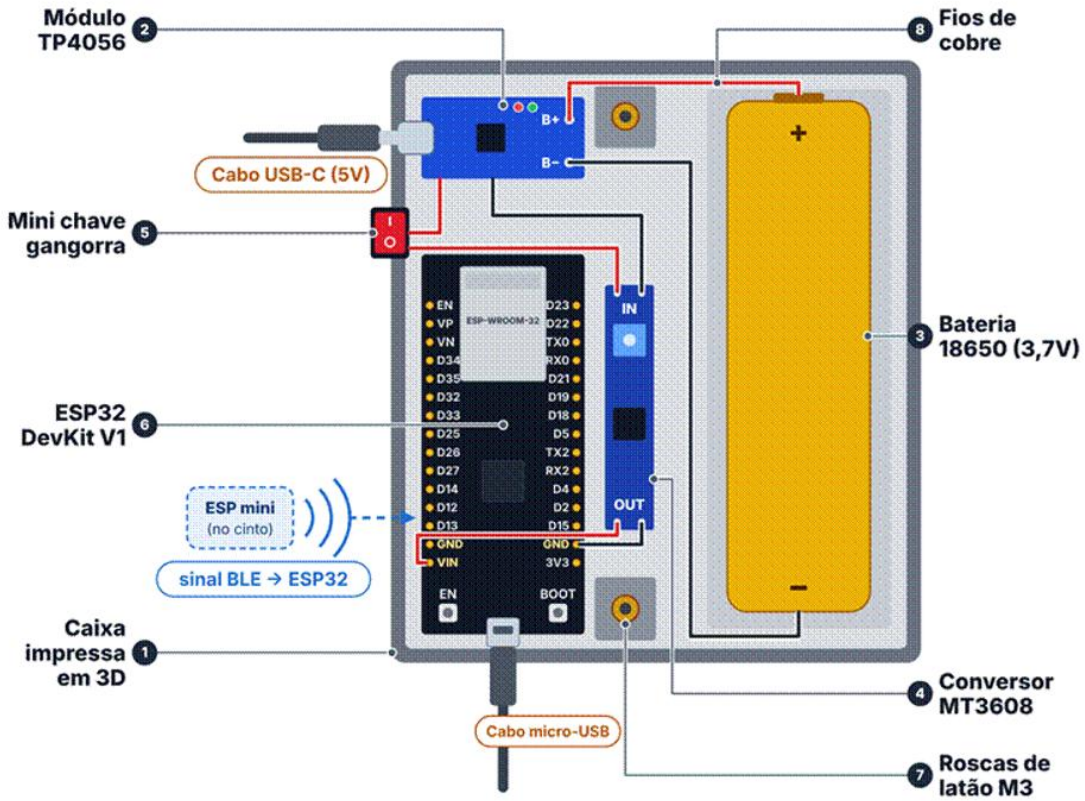
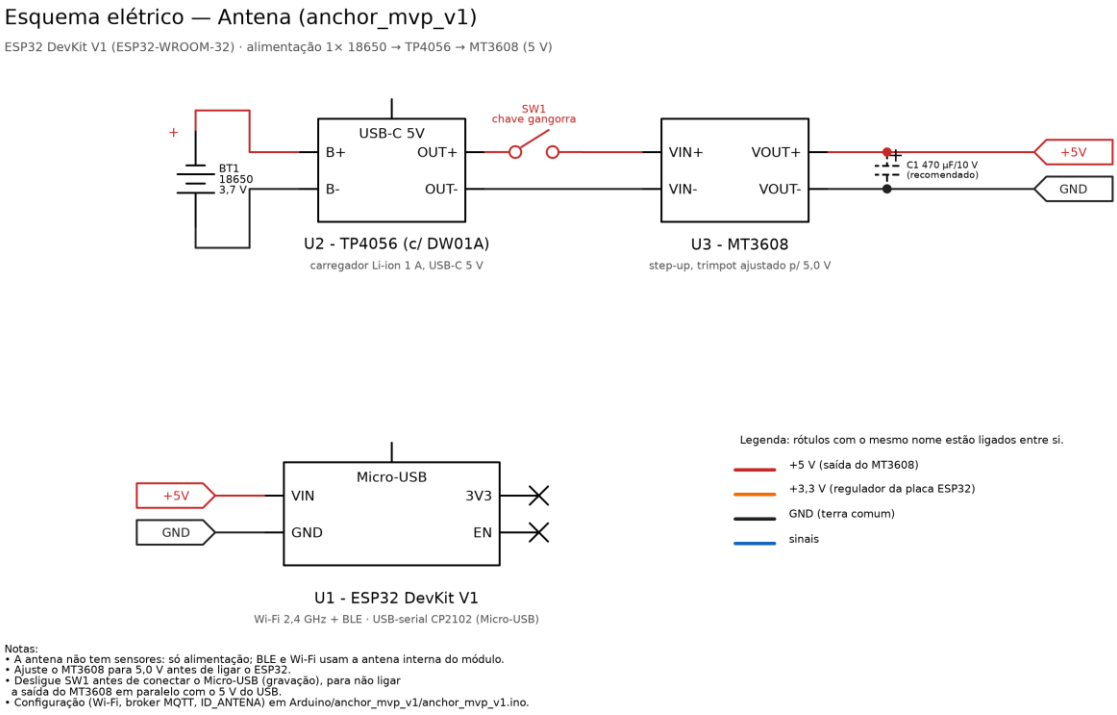


Figura 6 – Esquema elétrico da antena



Fonte das figuras: elaborado pelos autores (2026).

Componentes passivos:

- Pull-up do I²C: o módulo GY-521 já traz resistores de pull-up em SDA e SCL; não é preciso adicionar externos.
- Desacoplamento: os módulos (ESP32-C3, ESP32 DevKit, GY-521, AD8232, MT3608 e TP4056) já trazem capacitores de desacoplamento na própria placa. O capacitor C1 (470 µF/10 V) na saída do MT3608 é recomendado (tracejado no esquema) para absorver os picos de corrente do rádio, sobretudo na antena.
- LO+ / LO−: saídas digitais do próprio AD8232; dispensam resistores.
- SDN do AD8232: fica aberto; o módulo o mantém em nível alto (ligado).
- R1 (330 Ω): limita a corrente do LED opcional, calculada por

$$I_{LED} = \frac{V_{GPIO} - V_{LED}}{R_1} = \frac{3{,}3 - 2{,}0}{330} \approx 4 \text{ mA}$$

em que I_{LED} é a corrente no LED (A), V_{GPIO} é a tensão do pino em nível alto (3,3 V), V_{LED} é a queda de tensão do LED (cerca de 2,0 V) e R_1 é a resistência (Ω).

2.4 Especificações de alimentação

Tensões

Item	Cinto	Antena
Baterias	2 × 18650 em paralelo, 3,7 V nominal (3,0–4,2 V)	1 × 18650, 3,7 V nominal (3,0–4,2 V)
Saída do MT3608	5,0 V	5,0 V
Trilho de 3,3 V	Regulador linear da placa ESP32-C3 (alimenta MPU6050 e AD8232)	Regulador AMS1117 do DevKit (só o ESP32)
Carga (TP4056)	1 A, corte em 4,2 V	1 A, corte em 4,2 V

Consumo por componente (valores adotados pela equipe; o do ESP32 é o de pior caso)

Dispositivo	Componente	Corrente
Cinto	ESP32-C3 Super Mini	350 mA
Cinto	MPU6050 (acelerômetro + giroscópio)	5 mA
Cinto	AD8232 (ECG)	0,17 mA (170 µA)
Cinto	Total	≈ 355 mA
Antena	ESP32 DevKit V1	350 mA
Antena	Total	350 mA

Os sensores recebem 3,3 V de um regulador linear, que deixa passar a mesma corrente que recebe; por isso o total vale também para a linha de 5 V.

Caminho da energia

```
Bateria 3,7 V → TP4056 (proteção) → chave → MT3608 (sobe para 5 V) → regulador da placa (de
sce para 3,3 V) → ESP32 + sensores
```

A energia passa por duas conversões, e cada uma perde uma parte. O MT3608 é um conversor chaveado que eleva a tensão com cerca de 85% de eficiência. O regulador da placa é linear: deixa passar a mesma corrente que recebe e transforma em calor a diferença de tensão (5 V − 3,3 V).

Por que não basta dividir capacidade por consumo: a conta direta ($\frac{2000}{355} \approx 5{,}6$ h no cinto; $\frac{1000}{350} \approx 2{,}9$ h na antena) compara a capacidade da bateria, que trabalha em 3,7 V, com a corrente do circuito, que trabalha em 5 V. Ela ignora a elevação de tensão, as perdas do conversor e a parte da carga que não pode ser usada. Por isso o cálculo é feito em potência, nos passos a seguir.

Passo 1 – Potência na saída de 5 V

$$P_{5V} = V_{5V} \cdot I_{total}$$

em que:

- P_{5V} é a potência entregue ao circuito na saída do MT3608 (W);
- V_{5V} é a tensão de saída do MT3608 (5 V);
- I_{total} é a soma das correntes dos componentes (A).

Passo 2 – Potência retirada da bateria

$$P_{bat} = \frac{P_{5V}}{\eta}$$

em que:

- P_{bat} é a potência que sai da bateria (W);
- η é a eficiência do MT3608 (0,85, ou seja, 85%).

Passo 3 – Corrente na bateria

$$I_{bat} = \frac{P_{bat}}{V_{bat}}$$

em que:

- I_{bat} é a corrente retirada da bateria (A);

- V_{bat} é a tensão da bateria (3,7 V nominal; cerca de 3,3 V quando está quase vazia).

Grandeza	Cinto	Antena
I_{total} (5 V)	0,355 A	0,350 A
P_{5V}	5 × 0,355 = 1,78 W	5 × 0,350 = 1,75 W
P_{bat}	1,78 / 0,85 = 2,09 W	1,75 / 0,85 = 2,06 W
I_{bat} (bateria em 3,7 V)	2,09 / 3,7 ≈ 565 mA	2,06 / 3,7 ≈ 556 mA
I_{bat} com bateria quase vazia (3,3 V)	2,09 / 3,3 ≈ 633 mA	2,06 / 3,3 ≈ 624 mA

A corrente na bateria (565 mA) é maior que a do circuito (355 mA) porque a bateria tem tensão menor que a saída: para entregar a mesma potência, ela precisa fornecer mais corrente e ainda cobrir a perda do conversor. À medida que a bateria descarrega, a tensão cai e a corrente sobe. Essas correntes ficam bem abaixo do limite do MT3608 (chave interna de 4 A). No cinto, elas se dividem entre as duas células em paralelo, cerca de 0,3 A em cada.

Autonomia

Passo 1 – Energia armazenada

$$E = n \cdot C \cdot V_{bat}$$

em que:

- E é a energia nominal armazenada (Wh);
- n é o número de baterias;
- C é a capacidade de cada bateria (Ah; 1000 mAh = 1,0 Ah);
- V_{bat} é a tensão nominal da bateria (3,7 V).

Passo 2 – Energia útil

$$E_{útil} = E \cdot f_u$$

em que:

- $E_{útil}$ é a energia que pode ser usada antes do corte de proteção (Wh);
- f_u é a fração útil da carga (0,9), porque o circuito de proteção do TP4056 desliga a bateria perto de 2,5 V, antes de ela esvaziar totalmente.

Passo 3 – Tempo de funcionamento

$$t = \frac{E_{útil}}{P_{bat}}$$

em que:

- t é a autonomia (h);
- P_{bat} é a potência retirada da bateria, calculada acima (W).

Dispositivo	Baterias	Energia nominal	Energia útil	P_{bat}	Autonomia
Cinto	2 × 1000 mAh em paralelo	2 × 1,0 × 3,7 = 7,4 Wh	7,4 × 0,9 = 6,66 Wh	2,09 W	6,66 / 2,09 ≈ 3,19 h (3 h 11 min)
Antena	1 × 1000 mAh	1 × 1,0 × 3,7 = 3,7 Wh	3,7 × 0,9 = 3,33 Wh	2,06 W	3,33 / 2,06 ≈ 1,62 h (1 h 37 min)

A conversão para minutos usa a parte decimal: 0,19 h × 60 ≈ 11 min e 0,62 h × 60 ≈ 37 min.

Conferência em mAh: a mesma autonomia sai dividindo a capacidade útil pela corrente na bateria.

$$t = \frac{n \cdot C_{mAh} \cdot f_u}{I_{bat}}$$

em que C_{mAh} é a capacidade de cada bateria em mAh e I_{bat} é a corrente na bateria em mA. Para o cinto: 2 × 1000 × 0,9 = 1800 mAh; 1800 ÷ 565 ≈ 3,19 h. Para a antena: 1000 × 0,9 = 900 mAh; 900 ÷ 556 ≈ 1,62 h. Os dois métodos dão o mesmo resultado.

Como os 350 mA do ESP32 são um valor de pior caso, a autonomia real tende a ser maior que a calculada. Para sessões longas, deixe as antenas no carregador USB-C: o TP4056 alimenta o circuito enquanto carrega.

Tempo de recarga

O TP4056 carrega em duas fases. Na fase de corrente constante (CC), aplica 1 A até a bateria chegar a 4,2 V, o que enche cerca de 80% da carga. Na fase de tensão constante (CV), mantém 4,2 V enquanto a corrente cai até cerca de 100 mA, quando a carga termina. Como a fase CV é lenta, acrescenta-se cerca de 20% ao tempo:

$$t_{carga} \approx k \cdot \frac{n \cdot C}{I_{carga}}$$

em que:

- t_{carga} é o tempo de recarga (h);

- k é o fator da fase de tensão constante (1,2);
- n é o número de baterias;
- C é a capacidade de cada bateria (Ah);
- I_{carga} é a corrente de carga do TP4056 (1 A).

Dispositivo	Cálculo	Tempo de recarga
Cinto (2 × 1000 mAh)	1,2 × 2,0 Ah / 1 A	≈ 2,4 h (2 h 24 min)
Antena (1 × 1000 mAh)	1,2 × 1,0 Ah / 1 A	≈ 1,2 h (1 h 12 min)

Recarga com o aparelho ligado: o TP4056 entrega no máximo 1 A no total, e parte dessa corrente vai para o circuito. No cinto, o circuito consome cerca de 0,565 A, então sobram 1 − 0,565 ≈ 0,44 A para a bateria, e o tempo passa a ser $1{,}2 \times \frac{2{,}0}{0{,}44} \approx 5{,}5$ h, mais que o dobro. Na antena, 1 A equivale a 1C para uma célula de 1000 mAh; confira se a célula aceita essa corrente de carga.

Validação na bancada: para confirmar os valores, ligue o multímetro (escala de A) em série entre o polo positivo da bateria e o B+ do TP4056, com o dispositivo em operação normal, anote a corrente média e substitua esse I_{bat} medido na fórmula de conferência em mAh.

Dissipação térmica

TP4056 durante a carga (carregador linear):

$$P_{TP4056} = (V_{USB} - V_{bat}) \cdot I_{carga}$$

em que P_{TP4056} é a potência transformada em calor no TP4056 (W), V_{USB} é a tensão da USB (5 V), V_{bat} é a tensão da bateria naquele momento (V) e I_{carga} é a corrente de carga (1 A).

MT3608 durante o uso:

$$P_{MT3608} = P_{bat} - P_{5V}$$

em que P_{MT3608} é a perda no conversor (W), P_{bat} é a potência que sai da bateria e P_{5V} é a potência entregue em 5 V.

Regulador de 3,3 V da placa durante o uso (regulador linear):

$$P_{reg} = (V_{5V} - V_{3V3}) \cdot I_{total}$$

em que P_{reg} é a potência transformada em calor no regulador (W), V_{5V} é a tensão de entrada (5 V), V_{3V3} é a tensão de saída (3,3 V) e I_{total} é a corrente do circuito (A).

Componente	Cálculo	Potência dissipada	Observação
TP4056 (meio da carga)	(5 − 3,7) V × 1 A	≈ 1,3 W	Componente mais quente; o chip reduz a corrente sozinho se esquentar demais
TP4056 (início, bateria vazia em 3,0 V)	(5 − 3,0) V × 1 A	≈ 2,0 W	Pior momento da carga
MT3608	2,09 − 1,78 (cinto); 2,06 − 1,75 (antena)	≈ 0,31 W	Morno; manter folga no case
Regulador 3,3 V da placa	1,7 × 0,355 (cinto); 1,7 × 0,350 (antena)	≈ 0,60 W	Aquece; a tampa perfurada da antena ajuda a ventilar

O aquecimento de cada componente pode ser estimado por

$$\Delta T = P \cdot \theta_{JA}$$

em que ΔT é o aumento de temperatura acima do ambiente (°C), P é a potência dissipada no componente (W) e θ_{JA} é a resistência térmica do encapsulamento ao ar (°C/W, informada na folha de dados). No regulador do DevKit, com θ_{JA} entre 60 e 90 °C/W, 0,60 W resultam em 36 a 54 °C acima do ambiente. O regulador do ESP32-C3 Super Mini é menor e esquentaria demais se o consumo fosse realmente 350 mA o tempo todo, o que reforça que esse valor é de pico e que a medição deve indicar consumo menor.

Para onde vai a energia do cinto: dos 2,09 W retirados da bateria, cerca de 1,17 W (3,3 V × 0,355 A, 56%) são usados pelo ESP32 e pelos sensores, 0,60 W (29%) viram calor no regulador de 3,3 V e 0,31 W (15%) viram calor no MT3608. Quase metade da energia é perdida nas duas conversões (3,7 V → 5 V → 3,3 V); uma melhoria futura seria alimentar o circuito direto em 3,3 V a partir da bateria, com um regulador de baixa queda, sem passar pelos 5 V.

Ajuste do MT3608 (obrigatório antes de ligar o ESP32): com o ESP32 desconectado, alimente o MT3608 e gire o trimpot até ler 5,0 V na saída com o multímetro. Sem esse ajuste, a saída pode passar de 5 V e danificar a placa.

Cuidados térmicos e elétricos:

- O TP4056 aquece bastante durante a carga (~1,3 W). Mantenha folga entre ele e a bateria dentro do case e não carregue o dispositivo exposto ao sol, sobre superfícies quentes ou vestido pelo usuário.
- Só ligue as duas células do cinto em paralelo se tiverem a mesma tensão (diferença < 0,1 V); do contrário, uma carrega a outra com corrente alta.
- Use apenas células 18650 de procedência conhecida e sem amassados; descarte células estufadas.

2.5 Ordem de montagem

Arquivos de impressão 3D: os moldes dos cases ficam no repositório do projeto (Seção 3.1), na pasta `impressao_3d/`, separados em `impressao_3d/cinto/` (base e tampa do case do cinto) e `impressao_3d/antena/` (base e tampa perfurada do case da antena). Basta abrir os arquivos no fatiador da impressora (por exemplo, Cura, PrusaSlicer ou Bambu Studio) e imprimir.

Ferramentas: ferro de solda e estanho, multímetro, alicate de corte e desencapador, tubos termorretráteis, fita dupla-face e cola quente, chave de fenda pequena (para o trimpot do MT3608 e os parafusos M3).

Regras de segurança durante a montagem:

- Solde sempre com as baterias fora do circuito; elas são as últimas a serem ligadas.
- Confira a polaridade (+ e −) com o multímetro antes de cada ligação de energia. Bateria invertida ou em curto aquece e pode estufar.
- Nunca encoste os terminais da bateria um no outro nem em ferramentas metálicas.

Cinto

1. Imprimir o case do cinto (base e tampa) e conferir se as aberturas da chave gangorra, das duas entradas USB-C e do conector P2 estão livres.
2. Instalar os insertos M3: com o ferro de solda (~220 °C), pressione as roscas de latão nos furos do case até ficarem rentes à superfície. Espere esfriar antes de rosquear.

3. Gravar o firmware antes de montar: com o ESP32-C3 solto, grave o `cinto_mvp_v1.ino` pela USB (Seção 3.2.4). Assim, se houver problema na gravação, a placa ainda está acessível.

4. Preparar as baterias: meça a tensão das duas células 18650. Elas só podem ser ligadas em paralelo se a diferença for menor que 0,1 V; se não for, carregue a de menor tensão antes.

5. Ajustar o MT3608: alimente a entrada do MT3608 com uma das baterias (ou uma fonte de 3,7 V), sem nada ligado na saída, e gire o trimpot até o multímetro marcar 5,0 V na saída.

6. Montar a alimentação (Figura 4), ainda sem as baterias:

   - TP4056 OUT+ → chave gangorra → MT3608 VIN+;
   - TP4056 OUT− → MT3608 VIN−;
   - MT3608 VOUT+ → pino 5V do ESP32-C3; MT3608 VOUT− → GND do ESP32-C3;
   - se for usar, o capacitor C1 de 470 µF entre VOUT+ e VOUT− (terminal positivo no VOUT+).

7. Ligar as baterias: una as duas células em paralelo (positivo com positivo, negativo com negativo) e solde ao B+ e B− do TP4056.

8. Testar a alimentação: ligue a chave e meça 5,0 V no pino 5V e 3,3 V no pino 3V3 do ESP32-C3. O LED da placa deve acender. Desligue a chave para continuar.

9. Soldar o MPU6050 (tabela da Seção 2.2): VCC → 3V3, GND → GND, SDA → GPIO 8, SCL → GPIO 9. Os pinos AD0, INT, XDA e XCL ficam sem ligação.

10. Soldar o AD8232: 3.3V → 3V3, GND → GND, OUTPUT → GPIO 4, LO− → GPIO 2, LO+ → GPIO 3. O pino SDN fica sem ligação. Posicione o conector P2 alinhado com a abertura do case.

11. LED de eletrodo solto (opcional): GPIO 10 → resistor de 330 Ω → ânodo do LED; cátodo do LED → GND.

12. Teste funcional antes de fechar: ligue a chave e o painel no computador (Seção 5.1). Confira se uma antena conecta por GATT, se a taxa

fica perto de 100 amostras/s, se a aceleração em repouso fica perto de 1 g e se o status do eletrodo muda ao conectar e soltar os eletrodos.

13. Isolar e fixar: cubra as soldas com termorretrátil e prenda as placas e as baterias com fita dupla-face ou cola quente. O MPU6050 deve ficar firme, sem folga. Deixe acessíveis o USB-C do TP4056, o USB-C do ESP32-C3, o conector P2 e a chave (Figura 3).

14. Fechar o case com os parafusos M3, sem apertar em excesso, e fazer o teste de carga: ligue um cabo USB-C no TP4056 e confira o LED de carga (vermelho carregando; azul ou verde quando completo).

Antena (repetir para as três unidades)

1. Imprimir o case da antena (base e tampa perfurada) e gravar ou etiquetar a identificação (`anchor01`, `anchor02` ou `anchor03`).

2. Instalar os insertos M3 como no cinto.

3. Gravar o firmware antes de montar: grave o `anchor_mvp_v1.ino` com o `ID_ANTENA` igual à etiqueta do case (Seção 3.2.3).

4. Ajustar o MT3608 para 5,0 V, como no passo 5 do cinto.

5. Montar a alimentação (Figura 6), ainda sem a bateria:
   - TP4056 OUT+ → chave gangorra → MT3608 VIN+;
   - TP4056 OUT− → MT3608 VIN−;
   - MT3608 VOUT+ → pino VIN do ESP32 DevKit; MT3608 VOUT− → GND do DevKit;
   - capacitor C1 de 470 µF entre VOUT+ e VOUT− (recomendado na antena, por causa dos picos de corrente do Wi-Fi).

6. Ligar a bateria ao B+ e B− do TP4056.

7. Testar: ligue a chave e meça 5,0 V no pino VIN; o LED de alimentação do DevKit deve acender. Com o painel aberto, a antena deve aparecer como online em até 10 s.

8. Isolar, fixar e fechar: termorretrátil nas soldas, placas e bateria presas com fita dupla-face ou cola quente, Micro-USB do ESP32, USB-C do TP4056 e chave acessíveis pelas aberturas (Figura 5). Feche a tampa com os parafusos M3.

2.6 Fixação mecânica e encapsulamento

1. Disposição interna: siga a Figura 3 (cinto) e a Figura 5 (antena). Prenda as baterias e as placas com fita dupla-face ou cola quente, deixando acessíveis:
    - Cinto: USB-C do TP4056 (carga), USB-C do ESP32-C3 (gravação), conector P2 do AD8232 e a chave gangorra.
    - Antena: USB-C do TP4056, Micro-USB do ESP32 e a chave gangorra.
2. Isolamento: cubra as soldas expostas com termorretrátil e garanta que nenhum terminal de bateria encoste em partes metálicas.
3. Fechamento: feche a tampa com os parafusos M3, sem apertar em excesso.
4. Fixação do cinto: o case vai preso ao cinto na frente do corpo, na altura da cintura, posição equivalente à do dataset SisFall usado no treino. O MPU6050 deve ficar firme no case, sem folga: vibração solta altera as leituras.
5. Fixação das antenas: instale cada antena em parede ou suporte, a cerca de 1 m do chão ou mais, longe de roteadores, micro-ondas e outras fontes de 2,4 GHz, e afastadas entre si. Anote a posição (x, y) de cada uma em metros para a configuração do ambiente (Seção 5.1). As três antenas não podem ficar alinhadas (colineares).
6. Ambiente: o protótipo é para uso interno; o case não é vedado contra água. Não há partes ligadas à rede elétrica; o único aterramento é o GND comum de baixa tensão de cada dispositivo.

SEÇÃO 3 — GUIA DE INSTALAÇÃO E CONFIGURAÇÃO DO SOFTWARE

3.1 Requisitos mínimos

Obtenção do projeto: todo o material do sistema está em um único repositório no GitHub, https://github.com/KaykyCrespo/cinto-inteligente-deteccao-quedas: código do painel e do backend, firmware do cinto e das antenas, bibliotecas Arduino usadas, modelo de IA treinado, moldes de impressão 3D dos cases (`impressao_3d/`), documentação e o arquivo `requirements.txt` com as dependências Python. Para baixar, use o botão Code → Download ZIP na página do repositório e descompacte, ou, com o Git instalado:

```
git clone https://github.com/KaykyCrespo/cinto-inteligente-deteccao-quedas.git
cd cinto-inteligente-deteccao-quedas
```

Item	Versão / requisito
Sistema operacional	Windows 10 ou 11, 64 bits
Python	3.12 ou superior (testado em 3.13)
Arduino IDE	2.3 ou superior
Pacote de placas ESP32	“esp32 by Espressif Systems” 3.x
Bibliotecas Arduino	NimBLE-Arduino 2.5.2, PubSubClient 2.8 (cópias em `Arduino/libraries/`)
Broker MQTT	Eclipse Mosquitto 2.x
Driver USB-serial	Silicon Labs CP210x (antenas); ESP32-C3 usa USB nativo (Seção 3.2.2)
Navegador	Chrome, Edge ou Firefox atualizado
Rede	Wi-Fi 2,4 GHz com o computador e as antenas na mesma sub-rede

Dependências Python (`requirements.txt`):

Pacote	Versão	Uso
streamlit	1.52.2	Painel web
paho-mqtt	2.1.0	Cliente MQTT
pandas	2.3.3	Tabelas e exportação
numpy	2.2.0	Cálculo dos atributos
scipy	1.14.1	Reamostragem no treino
scikit-learn	1.9.1	Random Forest
joblib	1.4.2	Carga do modelo
altair	6.0.0	Gráficos

3.2 Configuração da borda (firmware)

3.2.1 Instalar a Arduino IDE e o pacote ESP32

1. Baixe e instale a Arduino IDE 2.x em https://www.arduino.cc/en/software.
2. Em Arquivo → Preferências → URLs adicionais para Gerenciadores de Placas, adicione:

`https://espressif.github.io/arduino-esp32/package_esp32_index.json`

3. Em Ferramentas → Placa → Gerenciador de Placas, pesquise `esp32` e instale esp32 by Espressif Systems (3.x).

4. Bibliotecas: copie as pastas de `Arduino/libraries/` do repositório para `Documentos\Arduino\libraries\`. Outra opção é instalar pelo Gerenciador de Bibliotecas `NimBLE-Arduino` (h2zero) 2.5.2 e `PubSubClient` (Nick O'Leary) 2.8. Reinicie a IDE.

3.2.2 Drivers USB e reconhecimento da porta COM

Placa	Interface USB	Driver no Windows
Antena — ESP32 DevKit V1	Chip CP2102 (Silicon Labs)	Instale o CP210x Universal Windows Driver da página "CP210x USB to UART Bridge VCP Drivers" da Silicon Labs (descompacte, clique com o botão direito em `silabser.inf` → Instalar)
Antena — clone com chip CH340	CH340/CH341	Driver CH341SER da WCH
Cinto — ESP32-C3 Super Mini	USB nativo (USB Serial/JTAG)	Nenhum: o Windows 10/11 reconhece sozinho

Para conferir:

1. Conecte a placa com um cabo USB de dados (cabos só de carga não aparecem).

2. Abra o Gerenciador de Dispositivos → Portas (COM e LPT):

   - Antena: `Silicon Labs CP210x USB to UART Bridge (COMx).`
   - Cinto: `Dispositivo Serial USB (COMx).`

3. Se surgir um "Dispositivo desconhecido" com triângulo amarelo, instale o driver da tabela e reconecte.

4. Se o ESP32-C3 não aparecer: segure BOOT, conecte o cabo (ou aperte RESET) e solte BOOT. A placa entra em modo de gravação e a porta aparece.

Atenção: antes de conectar o USB para gravar, desligue a chave gangorra. Assim o 5 V do MT3608 não fica em paralelo com o 5 V do USB.

3.2.3 Configurar e gravar o firmware da antena

(`Arduino/anchor_mvp_v1/anchor_mvp_v1.ino`)

1. Configure o Wi-Fi em um arquivo local: na pasta `Arduino/anchor_mvp_v1/`, copie `credenciais_exemplo.h` para `credenciais.h` e preencha com a rede 2,4 GHz:

```
#define WIFI_SSID       "<nome da rede 2,4 GHz>"
#define WIFI_PASSWORD   "<senha da rede>"
```

O `credenciais.h` é ignorado pelo git, então a senha não vai para o repositório. Se ele não existir, o sketch compila com valores de exemplo e a antena não conecta ao Wi-Fi. Na Arduino IDE ele aparece como uma aba ao lado do sketch; o mesmo arquivo serve para as três antenas. Sempre que mudar a rede, grave o firmware de novo.

2. Abra o sketch e confira o topo do arquivo, ajustando o número da antena e o IP do broker:

```
#define CODIGO_AMBIENTE "OQPNB"          // igual ao do cinto
#define ID_ANTENA       1                // 1, 2 ou 3 (um por placa; igual à etiquet
a do case)
#define MQTT_BROKER     "<IPv4 do computador central>"
#define MQTT_PORT       1883
#define MQTT_USER       ""               // vazio: broker sem autenticação (rede loc
al)
#define MQTT_PASS       ""
#define MQTT_TOPIC_BASE "oqpnb/anchor"
```

O IPv4 do computador aparece com `ipconfig` no PowerShell (linha Endereço IPv4 do adaptador Wi-Fi).

3. Ferramentas → Placa → esp32 → ESP32 Dev Module (ou DOIT ESP32 DEVKIT V1).

4. Ferramentas → Porta → a COM da antena.

5. Clique em Carregar. Se aparecer `Connecting....___`, segure BOOT até começar a gravar.

6. Abra o Monitor Serial a 115200 baud. A cada 5 s a antena imprime o modo (`GATT+SENSORES` ou `SCAN_RSSI`), RSSI, contadores e estado do Wi-Fi/MQTT.

7. Repita para as antenas 2 e 3, mudando apenas `ID_ANTENA`.

Tópicos MQTT publicados pela antena:

Tópico	Conteúdo
`oqpnb/anchor/<ID_ANTENA>`	`{OQPNB;id_antena;rssi;id_cinto;ax;ay;az;gx;gy;gz;eletrodos;ecg}` (com GATT) ou `{OQPNB;id_antena;rssi;id_cinto;;;;;;;;}` (só RSSI)
`oqpnb/anchor/<ID_ANTENA>/status`	`online` / `offline` (mensagem retida, com Last Will)

3.2.4 Configurar e gravar o firmware do cinto (`Arduino/cinto_mvp_v1/cinto_mvp_v1.ino`)

1. Confira o topo do arquivo:

```
#define CODIGO_AMBIENTE "OQPNB"   // igual ao das antenas
#define ID_CINTO        1
#define FREQ_LEITURA_HZ 100       // deve ser igual à do modelo
```

Não altere as faixas do MPU em `iniciarMPU()` (±8 g / ±2000 °/s): o modelo foi treinado nelas.

2. Ferramentas → Placa → esp32 → ESP32C3 Dev Module.
3. Ferramentas → USB CDC On Boot → Enabled (sem isso o Monitor Serial fica vazio).
4. Selecione a porta e clique em Carregar.
5. No Monitor Serial (115200), o cinto informa quando uma antena conecta, quando o código de ambiente é validado e quando o anúncio alterna entre conectável e beacon.

Comunicação BLE do cinto:

Elemento	Valor
Serviço GATT	`FFF0`
`FFF1` (write)	A antena escreve o `CODIGO_AMBIENTE`; o cinto só transmite depois de validar (prazo de 5 s)
`FFF2` (notify)	Pacote a 100 Hz: `{OQPNB;id_cinto;ax;ay;az;gx;gy;gz;eletrodos;sinal_ad8232}`
Advertising	Manufacturer Data `{OQPNB;id_cinto}`; conectável a cada 20–40 ms sem GATT; beacon não conectável a cada 100 ms com GATT ativo

Uma antena por vez se conecta por GATT e repassa os sensores; as outras medem o RSSI dos anúncios.

3.3 Subida do servidor, backend e banco de dados

3.3.1 Broker Mosquitto

1. Instale o Mosquitto 2.x para Windows (https://mosquitto.org/download/), mantendo o serviço do Windows marcado.
2. Edite `C:\Program Files\mosquitto\mosquitto.conf` (Bloco de Notas como administrador) e adicione:

```
listener 1883 0.0.0.0
allow_anonymous true
```

Configuração para rede local de laboratório. Em produção, use `allow_anonymous false` com `password_file` e TLS na porta 8883.

3. Em um PowerShell como administrador:

```
Restart-Service mosquitto
New-NetFirewallRule -DisplayName "Mosquitto MQTT 1883" -Direction Inbound -Protocol
TCP -LocalPort 1883 -Action Allow -Profile Private,Domain
```

4. Em Configurações → Rede e Internet → Wi-Fi → (rede), marque a rede como Privada.

3.3.2 Backend Python

O repositório já traz o arquivo `requirements.txt`, que lista todas as bibliotecas Python do sistema com as versões exatas (tabela da Seção 3.1). Não é preciso instalar uma por uma: um único comando instala todas as dependências.

1. Instale o Python 3.12 ou superior em https://www.python.org/downloads/. Na primeira tela do instalador, marque Add python.exe to PATH.
2. Abra o PowerShell na pasta do projeto (no Explorador de Arquivos, abra a pasta, clique na barra de endereço, digite `powershell` e pressione Enter).
3. Crie e ative um ambiente virtual (recomendado, para não misturar com outras instalações do computador):

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Se o PowerShell bloquear a ativação, rode antes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e confirme.

4. Instale as dependências com um único comando:

```
pip install -r requirements.txt
```

O `pip` lê o `requirements.txt` e baixa todas as bibliotecas nas versões indicadas.

5. Valide a instalação rodando os testes automatizados; o resultado esperado é `OK`:

```
python -m unittest discover -s tests -p "teste_*.py" -v
```

O arquivo `requirements-docs.txt` traz dependências opcionais, usadas apenas para regenerar os diagramas e este manual (`pip install -r requirements-docs.txt`); não é necessário para operar o sistema.

3.3.3 Banco de dados

- Usa SQLite, sem servidor a instalar. O arquivo `dados/oqpnb.db` e as 7 tabelas (`cinto`, `antena`, `sessao`, `amostra`, `janela`, `queda`, `status_antena`) são criados

automaticamente por `banco.py` (`CREATE TABLE IF NOT EXISTS`) na primeira abertura do painel. Não há migração manual.

- O banco funciona em modo WAL e grava em lotes por uma thread dedicada. Os horários ficam em UTC (ISO-8601).
- O DER, os modelos lógico e físico e o dicionário de dados estão em `DOCUMENTACAO/README_BANCO_DE_DADOS.md`.

3.3.4 Variáveis de ambiente

O painel lê as variáveis de ambiente do processo. Modelo (`.env.example`):

```
# Telegram (obrigatórias para enviar alertas)
OQPNB_TG_TOKEN=<token gerado pelo @BotFather>
OQPNB_TG_CHAT=<chat_id do destinatário>

# MQTT (padrão: localhost:1883)
OQPNB_BROKER=localhost
OQPNB_PORT=1883

# Caminhos (padrão: pastas do projeto)
OQPNB_MODELO=modelos/modelo_mvp_8g.joblib
OQPNB_DB=dados/oqpnb.db

# Localização (padrão do ensaio de calibração)
OQPNB_LARGURA_M=5.0
OQPNB_ALTURA_M=5.0
OQPNB_RSSI_1M=-62
OQPNB_N_PERDA=2.0
OQPNB_OFFSET_GATT=0
```

Para definir no PowerShell, na mesma janela que abrirá o painel:

```
$env:OQPNB_TG_TOKEN = "<token>"
$env:OQPNB_TG_CHAT  = "<chat_id>"
```

Credenciais fora do repositório: o token do bot, o chat ID e a senha do Wi-Fi nunca ficam no código versionado. O token e o chat ID podem ser definidos pelas variáveis de ambiente acima ou no arquivo local `segredos.env`, na raiz do projeto: copie `segredos.env.exemplo` para `segredos.env` e preencha `OQPNB_TG_TOKEN` e `OQPNB_TG_CHAT`. Esse arquivo é ignorado pelo git e lido automaticamente ao abrir o painel; variáveis já definidas no ambiente têm prioridade sobre ele. A senha do Wi-Fi das antenas fica em `credenciais.h` (Seção 3.2.3). Em nenhuma hipótese publique tokens e senhas reais em repositórios ou arquivos versionados.

Como criar o bot do Telegram e obter o token e o chat ID

O alerta de queda é enviado por um bot do Telegram. São necessários dois dados: o token (a chave de acesso do bot à API do Telegram) e o chat ID (o identificador da conversa que vai receber os alertas).

Parte A – Criar o bot e obter o token:

1. Instale o Telegram no celular ou no computador e entre com a sua conta.
2. Na busca, procure por @BotFather (conta oficial do Telegram, com selo azul de verificação) e abra a conversa.
3. Toque em Iniciar (ou envie `/start`) e depois envie o comando `/newbot`.
4. O BotFather pede o nome de exibição do bot. Envie, por exemplo, `Alerta Cinto Inteligente`.
5. Em seguida, ele pede o nome de usuário do bot, que deve ser único e terminar em `bot`. Envie, por exemplo, `cinto_inteligente_alerta_bot`. Se o nome já existir, tente outro.
6. O BotFather responde com a mensagem *"Use this token to access the HTTP API"*, seguida do token, no formato `123456789:ABCdefGhIJKlmNoPQRstuVWxyz`. Copie e guarde esse valor: ele é o `OQPNB_TG_TOKEN`.
7. Trate o token como uma senha. Se ele for exposto, gere um novo no BotFather com o comando `/revoke`, escolha o bot e atualize a variável.

Parte B – Obter o chat ID:

1. Na mesma resposta, o BotFather envia o link do bot (`t.me/<nome_de_usuário>`). Abra o link, toque em Iniciar e envie qualquer mensagem, por exemplo `oi`. Esse passo é obrigatório: um bot não pode iniciar uma conversa, só responder a quem já falou com ele.
2. No navegador, acesse o endereço abaixo, trocando `<TOKEN>` pelo token da Parte A (sem os sinais `<` e `>`):

   `https://api.telegram.org/bot<TOKEN>/getUpdates`
3. A página mostra um texto em formato JSON. Procure o trecho `"chat":{"id":123456789, ...}`. O número após `"id":` é o chat ID, que vai em `OQPNB_TG_CHAT`.
4. Se aparecer apenas `{"ok":true,"result":[]}`, a mensagem ainda não chegou: envie outra mensagem ao bot e recarregue a página.

5. Para enviar os alertas a um grupo (por exemplo, toda a equipe de cuidado): crie o grupo, adicione o bot como membro, envie uma mensagem no grupo mencionando o bot (`@cinto_inteligente_alerta_bot oi`) e repita os passos 2 e 3. O chat ID de grupo é negativo (por exemplo, `-1001234567890`) e deve ser usado com o sinal de menos.

Parte C – Configurar e testar:

1. Copie `segredos.env.exemplo` para `segredos.env`, na raiz do projeto, e preencha:

```
OQPNB_TG_TOKEN=<token da Parte A>
OQPNB_TG_CHAT=<chat ID da Parte B>
```

Outra opção é definir as variáveis no PowerShell que vai abrir o painel (`$env:OQPNB_TG_TOKEN = "<token>"` e `$env:OQPNB_TG_CHAT = "<chat ID>"`) ou de forma permanente com `setx`, que só vale para as janelas abertas depois do comando.

2. Abra o painel (`python -m streamlit run app.py`). Na barra lateral, a opção Enviar alerta de quedas fica habilitada.

3. Clique em Testar envio. O bot deve enviar a mensagem "☑ Teste do Cinto Inteligente" com a data e a hora. Se aparecer erro, consulte a Seção 5.3 (erros 401, 403 e 404 do Telegram).

3.4 Aplicação cliente (painel)

```
python -m streamlit run app.py
```

O painel abre em http://localhost:8501. Outros computadores da mesma rede acessam pelo `http://<IP do computador>:8501`.

Ferramentas auxiliares (opcionais):

```
# monitor em terminal
python interface_mvp_2.py --modelo modelos/modelo_mvp_8g.joblib --quiet

# reprocessar uma gravação sem hardware
python scripts/reproduzir_csv.py dados/gravacoes/<arquivo>.csv --modelo modelos/modelo_mvp_
8g.joblib
```

SEÇÃO 4 — MÓDULO DE INTELIGÊNCIA ARTIFICIAL

4.1 Objetivo operacional

A IA faz classificação binária supervisionada de janelas de movimento: para cada janela de 2,5 s do sensor inercial, estima a probabilidade de possível queda em relação a atividade normal. Se a probabilidade passa do limiar (padrão 0,50, ajustável no painel), o sistema registra a queda, mostra o alerta no painel e envia uma mensagem pelo Telegram. A localização por RSSI e o ECG não entram no modelo; servem de contexto para o alerta.

4.2 Dataset de treinamento e validação

Item	Descrição
Origem	Dataset público SisFall (Sucerquia; López; Vargas-Bonilla, 2017), em `dados/SISFALL/`
Participantes	38 voluntários (23 adultos jovens SA01–SA23 e 15 idosos SE01–SE15)
Registros	1.798 quedas simuladas (F01–F15) e 2.707 atividades da vida diária (D01–D19)
Sensor usado	Acelerômetro ADXL345 (±16 g) e giroscópio ITG3200 (±2000 °/s) presos à cintura, a 200 Hz
Volume após janelamento	122.572 janelas (28.768 de queda e 93.804 de atividade)

Pré-processamento (`scripts/treino_sisfall.py`):

1. Conversão das contagens do SisFall para g e °/s.
2. Reamostragem de 200 Hz para 100 Hz, a taxa do cinto.
3. Simulação do sensor do cinto: saturação em ±8 g / ±2000 °/s, as mesmas faixas configuradas no MPU6050.
4. Janelamento: janelas de 2,5 s com passo de 0,5 s. Nos registros de queda, entram como positivas as janelas que contêm o impacto.

Engenharia de atributos (`features.py`, a mesma função no treino e na operação). O uso de magnitudes torna os atributos independentes da orientação do sensor, o que dispensa normalização adicional:

Atributo	Definição	Importância no modelo
`tilt`	Ângulo entre a gravidade no início e no fim da janela (graus)	0,33
`gstd`	Desvio padrão de |ω| (°/s)	0,18
`amax`	Máximo de |a| (g)	0,13
`gmax`	Máximo de |ω| (°/s)	0,12
`astd`	Desvio padrão de |a| (g)	0,08

Atributo	Definição	Importância no modelo
`jerk`	Máxima variação de |a| por segundo (g/s)	0,07
`post_std`	Desvio de |a| no último 0,5 s (imobilidade após o impacto)	0,04
`amin`	Mínimo de |a| (queda livre)	0,04

4.3 Arquitetura do modelo e treinamento

Algoritmo: Random Forest (`sklearn.ensemble.RandomForestClassifier`).

Por que Random Forest e não as alternativas:

- Regra de limiar (pico de aceleração): simples, mas gera muitos falsos positivos em atividades bruscas (sentar rápido, tropeçar).
- Redes neurais (CNN/LSTM): exigem mais dados, mais tempo de treino e mais poder de inferência, e são menos interpretáveis.
- Random Forest: funciona bem com poucos atributos tabulares, não exige normalização, tolera ruído, informa a importância de cada atributo e tem inferência de milissegundos em CPU comum. Está entre os classificadores mais usados na detecção de quedas com sensores vestíveis.

Hiperparâmetros:

Parâmetro	Valor	Motivo
`n_estimators`	200	Estabilidade da votação
`min_samples_leaf`	2	Reduz o sobreajuste
`class_weight`	`balanced_subsample`	Compensa o desbalanceamento (23% de janelas de queda)
Limiar de decisão	0,50	Equilíbrio entre sensibilidade e especificidade (Seção 4.4)

Validação: validação cruzada por voluntário (GroupKFold, 5 folds). Nenhum voluntário aparece ao mesmo tempo no treino e no teste. Isso evita o vazamento causado por janelas sobrepostas da mesma pessoa e simula o uso por alguém que o modelo nunca viu. O modelo final é treinado com todos os dados e salvo em `modelos/modelo_mvp_8g.joblib`.

Para retreinar:

```
python scripts/treino_sisfall.py --sisfall dados/SISFALL/SisFall_dataset --saida modelos/modelo_mvp_8g.joblib
```

4.4 Resultados e métricas

As métricas são calculadas por registro: um registro de queda conta como detectado se alguma janela passar do limiar; um registro de atividade conta como falso alarme se alguma janela passar do limiar. O relatório completo está em `modelos/resultado_treino_mvp_8g.txt`.

Matriz de confusão (validação cruzada por voluntário, limiar 0,50):

	Previsto: queda	**Previsto: atividade**
Real: queda (1.798)	VP = 1.777	FN = 21
Real: atividade (2.707)	FP = 128	VN = 2.579

Métrica	**Fórmula**	**Valor**
Revocação (sensibilidade)	VP / (VP + FN)	98,8%
Especificidade	VN / (VN + FP)	95,3%
Precisão	VP / (VP + FP)	93,3%
F1-score	2·P·R / (P + R)	96,0%
Acurácia	(VP + VN) / total	96,7%

em que VP (verdadeiros positivos) são quedas detectadas como queda; FN (falsos negativos) são quedas não detectadas; FP (falsos positivos) são atividades normais que geraram alarme; VN (verdadeiros negativos) são atividades normais sem alarme; P é a precisão; R é a revocação; e total é a soma VP + FN + FP + VN (4.505 registros).

Sensibilidade por fold: 96,8% · 98,1% · 100% · 100% · 99,5%. Especificidade por fold: 96,3% · 94,8% · 97,4% · 94,2% · 93,6%.

Efeito do limiar (configurável no painel):

Limiar	**Sensibilidade**	**Especificidade**
0,3	99,2%	91,9%
0,4	99,0%	94,0%
0,5	98,8%	95,3%
0,6	98,4%	96,5%
0,7	97,9%	97,8%
0,8	97,0%	98,6%

Limitações conhecidas:

- A maior fonte de falsos alarmes é a atividade D11 (tentar levantar e desabar na cadeira: 53/190), seguida de D06 (subir e descer escadas rápido), D14 (mudar de posição deitado), D18 (tropeçar) e D19 (saltar). São movimentos de fato parecidos com uma queda.
- As quedas do SisFall foram simuladas, em sua maioria por adultos jovens; o desempenho com idosos reais em uso contínuo não foi medido.

- O modelo não aprende em operação; incorporar novos dados exige retreino.
- Após um alarme, o mesmo cinto fica 5 s sem gerar novo alarme, para evitar alertas duplicados.

4.5 Declaração formal de uso de IA assistiva

Foram usadas ferramentas de IA generativa (modelos de linguagem) como apoio na confecção da documentação, da diagramação e do código, conforme a tabela abaixo. Nenhuma ferramenta de IA é autora ou coautora do trabalho. Todo o conteúdo gerado foi revisado, testado e validado pelos integrantes, que assumem inteira responsabilidade técnica, ética e legal pelos artefatos entregues. Não foram inseridos nas ferramentas dados pessoais ou sensíveis de participantes.

Ferramenta	Versão e ambiente	Período	Finalidade	Intervenção humana
Claude (Anthropic)	claude-opus-5-5, usado no aplicativo	14/09/2026 a 12/11/2026	Organização do material do grupo, revisão ortográfica, apoio para as normas ABNT e criação de diagramas e tabelas	Todas as partes do texto foram revisadas, com taxa de correção de 80% sobre todo o documento
Claude (Anthropic)	Sonnet 4.6 low, usado no aplicativo	14/09/2026 a 12/11/2026	Desenvolvimento do sistema web e ajustes no código dos eletrônicos	Código inteiramente revisado pelos integrantes; refatoração de cerca de 75% do código
ChatGPT (OpenAI)	ChatGPT 5.6 Sol extra alto	14/10/2026 a 12/11/2026	Desenvolvimento do sistema web e ajustes no código dos eletrônicos	Código inteiramente revisado pelos integrantes; refatoração de cerca de 75% do código

Todos os prompts e as saídas brutas estão arquivados pela equipe e podem ser disponibilizados ao orientador e à banca, se solicitado. A declaração completa, assinada, está no Apêndice A do TCC.

O modelo de Random Forest do produto não é IA generativa: foi treinado pela equipe com o script `scripts/treino_sisfall.py` sobre o dataset público SisFall.

SEÇÃO 5 — MANUAL DE OPERAÇÃO E DIAGNÓSTICO

5.1 Partida e calibração

Ordem de partida:

1. Computador: confirme que o serviço Mosquitto está em execução (`Get-Service mosquitto → Running`) e que o Wi-Fi está conectado à mesma rede das antenas.
2. Painel: abra o PowerShell na pasta do projeto, ative o ambiente virtual, defina as variáveis do Telegram e rode `python -m streamlit run app.py`.
3. Antenas: ligue a chave gangorra das três antenas. Em até ~10 s, cada uma deve aparecer como online na aba Detalhes → Status do Sistema.
4. Eletrodos: coloque os três eletrodos (RA, LA e RL) na pele limpa e seca e encaixe o cabo P2 no cinto.
5. Cinto: ligue a chave gangorra. Uma antena se conecta por GATT e o cartão do cinto aparece na aba Monitoramento com taxa próxima de 100 amostras/s.

Verificação do handshake:

Ponto	Onde verificar	Esperado
Antena ↔ Wi-Fi/MQTT	Monitor Serial da antena; painel → Detalhes	Linha `[STATUS] ... WiFi: ... MQTT: ...` sem falhas e `TX MQTT` crescendo; antena online
Cinto ↔ antena (GATT)	Monitor Serial da antena; painel → Detalhes → antenas	`[GATT] CONECTADO E AUTENTICADO!` e modo `GATT+SENSORES` em uma antena
Código de ambiente	Monitor Serial do cinto	Código validado
Fluxo de dados	Painel → Monitoramento	Amostras subindo, ~100 Hz
Eletrodos	Painel → Status Eletrodo; LED do cinto	“Conectado”; LED apagado

Calibração de repouso dos sensores:

1. Com o cinto parado sobre a mesa por 10 s, abra Detalhes → Gráficos de Sinais. A magnitude da aceleração deve ficar próxima de 1 g (gravidade) e o giroscópio próximo de 0 °/s.
2. O modelo usa magnitudes e variações (Seção 4.2), por isso não depende de offset fino do MPU6050. Se o repouso se afastar muito de 1 g, verifique a fixação do sensor e as faixas configuradas no firmware.

3. ECG: com o usuário sentado e parado, o traçado deve mostrar complexos QRS regulares. O ECG é experimental e sem finalidade médica.

Configuração do ambiente (localização):

1. Na aba Detalhes, abra o expansor Ambiente ao lado do mapa.
2. Informe a largura e a altura do cômodo em metros e as coordenadas (x, y) de cada antena, medidas a partir do canto inferior esquerdo.
3. Os valores ficam salvos em `dados/ambiente.json`. Se as antenas estiverem alinhadas, o painel avisa: reposicione-as.

Calibração do RSSI (recomendada a cada novo ambiente):

1. Coloque o cinto, vestido, a 1 m de uma antena, de frente para ela.
2. Na barra lateral, clique em Gravar amostras por ~60 s e depois em Parar gravação.
3. Calcule a mediana do RSSI dessa antena no CSV de `dados/gravacoes/`. Esse é o $RSSI_{1m}$ → `OQPNB_RSSI_1M`.
4. Repita a 2, 3 e 5 m e estime o expoente de perda:

$$n_{perda} = \frac{RSSI_{1m} - RSSI_d}{10\log_{10} d}$$

em que n_{perda} é o expoente de perda do ambiente (adimensional), $RSSI_{1m}$ é a mediana do RSSI medida a 1 m (dBm), $RSSI_d$ é a mediana do RSSI medida à distância d (dBm) e d é a distância entre o cinto e a antena (m). O resultado vai em `OQPNB_N_PERDA` (valores típicos em ambiente interno: 1,6 a 3,3).

5. Reinicie o painel com as novas variáveis.

5.2 Operação diária

Barra lateral:

Controle	Função
Enviar alerta de quedas	Liga/desliga o envio ao Telegram (fica desabilitado se o token não estiver definido)
Testar envio	Envia uma mensagem de teste ao Telegram
Limiar de alarme	Sensibilidade do detector. Menor = detecta mais e gera mais falsos alarmes; maior = o oposto (Seção 4.4)
Tempo para resetar alerta	Por quanto tempo o aviso de queda fica na tela (1–5 min)
Atualizar automaticamente / Intervalo	Atualização da tela (0,5–5 s)

Controle	Função
Gravar amostras	Grava os dados brutos em CSV e no banco (arquivos divididos a cada 60 min)
Reconectar	Refaz a conexão MQTT sem fechar o painel

Aba Monitoramento:

- Cartão do cinto: em situação normal, mostra o status normal. Em POSSÍVEL QUEDA, mostra em destaque a probabilidade, data, hora, antena mais próxima, quadrante e a contagem regressiva do aviso.
- Métricas: amostras recebidas, taxa (amostras/s), antenas online, Status Eletrodo e Status Telegram.
- Probabilidade de queda: gráfico das 8 últimas janelas (0–100%). Picos que cruzam o limiar geram alarme.
- Quedas Detectadas: tabela com cada evento e o status de envio ao Telegram.

Aba Detalhes:

- Status do Sistema: modelo carregado, limiar, pacotes inválidos, amostras perdidas, janelas reiniciadas, total de quedas e cartões das antenas (GATT ou Advertising, RSSI, antena mais próxima).
- Tabela RSSI e Histórico de Quedas.
- Gráficos de Sinais: aceleração e giroscópio em tempo real.
- Mapa: um círculo por antena com o raio estimado (sólido para a mais próxima, tracejado para as outras) e a posição do cinto. Azul = posição confirmada; laranja = baixa confiança; cinza = estimativa bruta.
- ECG: traçado do sinal do AD8232.
- Exportação: CSV por sessão e o arquivo `.db`.

Como interpretar a localização: o resultado deve ser lido como região (quadrante) do ambiente, não como ponto exato. O corpo de quem usa o cinto atenua o sinal (~22 dB para a antena às costas e ~12 dB lateralmente, medidos a 1 m), o que distorce as distâncias.

Ao receber um alerta: confira o cartão do cinto e o quadrante indicado e entre em contato com a pessoa ou vá até o local. O sistema é de apoio e não substitui o acompanhamento presencial.

5.3 Resolução de problemas

Categoria	Sintoma	Causa provável	Procedimento corretivo
USB / gravação	Placa não aparece em Portas (COM)	Driver ausente ou cabo só de carga	Instale o driver CP210x ou CH340 (Seção 3.2.2); troque por cabo de dados
USB / gravação	`Failed to connect to ESP32: Timed out`	Placa fora do modo de gravação	Segure BOOT durante `Connecting...`; no ESP32-C3, segure BOOT ao conectar o cabo
USB / gravação	Monitor Serial do cinto vazio	`USB CDC On Boot` desativado	Ative-o e grave de novo
USB / gravação	`Sketch too big`	Partição padrão pequena para Wi-Fi + BLE	Ferramentas → Partition Scheme → Huge APP (3MB No OTA)
Rede	Antena não conecta ao Wi-Fi	Rede de 5 GHz, SSID/senha errados, `credenciais.h` ausente ou sinal fraco	Use rede de 2,4 GHz; confira `WIFI_SSID`/`WIFI_PASSWORD` em `Arduino/anchor_mvp_v1/credenciais.h` e grave o firmware de novo
MQTT	Antena com Wi-Fi conectado, mas `[MQTT] Conectando ao broker ... Falhou (rc=...)`	IP do broker errado, firewall ou listener ausente	Confira `MQTT_BROKER` com `ipconfig`; regra de firewall da porta 1883; `listener 1883 0.0.0.0`; rede como Privada
MQTT	Painel mostra Erro MQTT	Mosquitto parado	`Restart-Service mosquitto` e clique em Reconectar
MQTT	Antena oscila entre online e offline	Alimentação instável ou Wi-Fi fraco	Carregue a bateria, confira os 5,0 V do MT3608, aproxime do roteador
BLE	"Antena online, mas nenhum pacote de cinto chegou"	Cinto desligado, `CODIGO_AMBIENTE` diferente ou fora de alcance	Ligue o cinto; iguale o código nos dois firmwares; veja `[GATT] CONECTADO E AUTENTICADO!` no Serial da antena
Perda de pacotes	Taxa bem abaixo de 100 amostras/s e "amostras perdidas" crescendo	Interferência em 2,4 GHz, cinto longe da antena GATT, rede congestionada	Afaste as antenas de roteadores e micro-ondas; aproxime o cinto; reduza outros dispositivos na rede
Perda de pacotes	"Janelas reiniciadas" aumentando	Lacunas > 0,3 s no fluxo (troca de antena GATT)	Normal durante reconexões; se constante, siga o item anterior
Formato	"Pacotes com formato inválido"	Firmware de versão diferente do painel	Grave `cinto_mvp_v1` e `anchor_mvp_v1` deste repositório
Sensor	Status Eletrodo "Solto" / LED aceso	Eletrodo descolado, gel seco ou cabo P2 frouxo	Troque os eletrodos e encaixe o P2 até o fim
Sensor	ECG muito ruidoso ou saturado	Movimento, mau contato ou interferência de 60 Hz	Limpe a pele, use eletrodos novos, mantenha o usuário parado na verificação
Sensor	Aceleração em repouso longe	MPU solto no case ou faixa	Fixe o módulo; confirme ±8 g / ±2000 °/s em `iniciarMPU()`

Categoria	Sintoma	Causa provável	Procedimento corretivo
	de 1 g	alterada no firmware	
IA	Muitos falsos alarmes	Limiar baixo ou atividades bruscas	Aumente o limiar (0,6–0,7); veja a tabela da Seção 4.4
IA	Quedas não detectadas	Limiar alto, cinto mal posicionado ou faixas do MPU diferentes das do modelo	Reduza o limiar; cinto na cintura, à frente; retreine se mudar as faixas
IA	"Modelo indisponível"	Arquivo `.joblib` ausente ou versão do scikit-learn diferente	Confira `modelos/modelo_mvp_8g.joblib`; reinstale com `pip install -r requirements.txt`
Localização	Quadrante instável ou errado	Sombreamento corporal, antenas colineares, calibração ausente	Reposicione as antenas fora de linha; calibre o RSSI (Seção 5.1); interprete como região
Telegram	Toggle desabilitado	Token e chat ID não configurados (`segredos.env` ausente ou vazio e variáveis não definidas)	Preencha `OQPNB_TG_TOKEN` e `OQPNB_TG_CHAT` em `segredos.env` (Seção 3.3.4) e reabra o painel
Telegram	Erro 403 – bot was blocked by the user	O destinatário bloqueou o bot	No Telegram, abra o bot, desbloqueie e envie `/start`
Telegram	Erro 401 / 404	Token inválido ou revogado	Gere um novo token no @BotFather
Energia	ESP32 reinicia sozinho (brownout)	Bateria fraca ou MT3608 abaixo de 5 V	Carregue a bateria; reajuste o MT3608 para 5,0 V
Painel	`Port 8501 is already in use`	Outra instância do Streamlit aberta	Feche a outra janela ou use `--server.port 8502`

5.4 Desligamento seguro e manutenção preventiva

Desligamento:

1. Se houver gravação em andamento, clique em Parar gravação.
2. Desligue a chave gangorra do cinto e depois das antenas.
3. No PowerShell do painel, pressione Ctrl + C. O banco é gravado em lotes de no máximo 1 s, então os dados já estão salvos ao encerrar.
4. Retire os eletrodos do usuário e descarte-os (são de uso único).

Manutenção preventiva:

Frequência	Ação
A cada uso	Eletrodos novos; limpar a pele com álcool 70% antes de aplicar

Frequência	Ação
Diária	Recarregar cinto e antenas pelo USB-C do TP4056 (LED vermelho = carregando; azul/verde = carga completa)
Semanal	Limpar o case e o cabo P2 com pano levemente umedecido em álcool 70%, com o dispositivo desligado; nunca molhar
Semanal	Backup de `dados/oqpnb.db` e `dados/gravacoes/` com o painel fechado (ou pelo botão de exportação do `.db`)
Mensal	Abrir os cases e conferir soldas, fixação do MPU6050 e estado das baterias (sem estufamento); medir os 5,0 V do MT3608
A cada novo ambiente	Reconfigurar o ambiente e recalibrar o RSSI (Seção 5.1)
Ao atualizar o firmware	Gravar cinto e antenas com a mesma versão do repositório (Seção 3.2); manter o mesmo `CODIGO_AMBIENTE`
Ao atualizar o software	`git pull`; `pip install -r requirements.txt`; rodar os testes automatizados
Periodicamente	Trocar o token do Telegram se ele tiver sido exposto e nunca versioná-lo